

Synaptic Plasticity as Short-Term Memory

Hebbian writes turn a network's wiring into working memory with a fixed-size state — until interference breaks retrieval. Companion artifact: interactive explainer (single-file web app, all computation live). DataForge 2026, Pathway track.

The claim (one falsifiable sentence)

A network can store arriving associations in a fixed-size synaptic state through Hebbian writes — no gradient step and no new memory slot per token — but as associations accumulate, the read returns every stored value whose key overlaps the cue, so interference corrupts retrieval until recall fails. The artifact lets a learner reproduce both halves of this sentence in under 60 seconds (store one pair: perfect recall; store forty: the same probe fails).

The mechanism

write: $M_t = \lambda \cdot M_{t-1} + v k^T$ read: $r = M_t \cdot q$ closed form: $M_t q = \sum_i \lambda^{(t-i)} (k_i \cdot q) v_i$

$M \in \mathbb{R}^{(d \times d)}$ is the synaptic state: every entry is a synapse, and the state's shape never changes. Each arriving association (key k , value v) potentiates every synapse between their active units — one rank-one Hebbian write, "cells that fire together wire together." Reading with cue q returns a similarity-weighted sum of all stored values. Sparse, near-orthogonal keys make the matching term dominate at low load; the residual sum of cross-key overlaps is interference. In the artifact ($d = 24$, ~ 5 active units per key, seed 42) exact-match recall is 100% through ~ 10 stored pairs and falls below 50% by ~ 22 — nothing was deleted; with $\lambda = 1$ every pair is still fully present, and only the read degrades. Finite capacity despite constant memory is the linear-attention capacity limit identified by Schlag, Irie & Schmidhuber (ICML 2021): "endlessly adding new associations to a memory of finite size inevitably will reach a limit." Setting $\lambda < 1$ swaps the failure mode from interference to geometric decay — forgetting old context rather than corrupting it.

Why it matters now

A deployed Transformer carries context in a key-value cache that grows with every token; the alternative — a fixed-size state rewritten in place — is the active design space of linear attention (Gated Linear Attention, ICML 2024; RWKV-7, 2025), test-time-training layers (Sun et al., 2024) and fast-weight programmers, whose lineage traces to Schmidhuber (1992) and whose peer-reviewed primer (TMLR 2025) describes exactly this mechanism: fast weights are "synaptic weights [that] dynamically change over time as a function of input observations, and serve as short-term memory storage." Titans (2024) add surprise-gated test-time memorization. The shared trade: $O(1)$ memory and compute per token versus finite capacity and interference. Every 2024-2026 architecture below chooses a point on that trade-off; the toy makes the trade-off visible and breakable.

The roles of BDH and BDH-CQ

BDH (arXiv:2509.26507, 2025) is a brain-inspired post-Transformer architecture in which this mechanism is the working memory itself, not an add-on: "the working memory of BDH during inference entirely relies on synaptic plasticity with Hebbian learning using spiking neurons" (abstract). The paper reports monosemantic synapses that strengthen when BDH "hears or reasons about a specific concept," and sparse, positive activation vectors — the toy mirrors the sparse non-negative coding. BDH is not a state-space model in the Mamba sense; its GPU formulation is a separate ReLU-low-rank construction with linear attention. BDH-CQ (arXiv:2608.09888, 2026) is the demonstration-driven member of the family: inputs at inference "continuously update the model's recurrent memory" and queries are solved by latent-space iteration without verbalized intermediate reasoning — per the problem statement, state "accumulates additively per demonstration," which is precisely the toy's $\lambda = 1$ regime. A 150M-parameter configuration reports 29.5% pass@2 on the public ARC-AGI-1 evaluation at $\sim \$0.0007$ computed cost per task, stated to break the previously reported cost-accuracy Pareto frontier. This is the state-adaptation route to new tasks, opposed to the optimization route (HRM/TRM fine-tune weights on augmented demonstrations before each puzzle).

Landscape (what differs, mechanistically)

System	Context carrier	Updated at inference	Limit / cost
Transformer	KV cache (grows/token)	append-only	memory grows $O(t)$; little interference
GLA / RWKV-7	fixed matrix state	learned gated writes	interference (this toy's failure)
Titans / TTT	learned fast state	test-time gradient writes	learned forgetting; extra compute
BDH	synapses (Hebbian, spiking)	plasticity while reading	session memory; dev-reported
BDH-CQ	recurrent memory	additive per demonstration	measured on ARC-AGI-1 interventions

Evidence levels and limitations

BDH and BDH-CQ results cited above are developer-reported arXiv preprints, not peer-reviewed at the time of writing, and no deployment is claimed; the BDH-CQ companion repository names external evaluators (L. Kaiser; R. Kinas and R. Zhong, the latter two also co-authors) — evaluation, not independent audit. The toy's numbers are fully reproducible: fixed seeds, and a cross-implementation test checks that the browser's JavaScript core matches the NumPy reference to $1e-9$. Limitations: the toy's write/read rules are hand-set, not learned (frontier systems learn them); no spiking dynamics; $d = 24$; capacity crossing points are seed-dependent (the qualitative collapse is not); and this explainer makes no claim about BDH beyond its published statements. Continue with: arXiv 2102.11174 (fast-weight programmers), 2508.08435 (TMLR primer), 2509.26507 (BDH), 2608.09888 (BDH-CQ).